TUGAS MATA KULIAH STRATEGI OPTIMASI KOMPUTASI AWAN

TASK 6



Oleh:

5027241050 Daniswara Fausta Novanto

5027241103 Ni`mah Fauziyyah Atok

5027241042 Abiyyu Raihan Putra Wikanto

5027241082 Muhammad Rafi` Adly

5027231043 Adi Satria Pangestu

DEPARTEMEN TEKNOLOGI INFORMASI INSTITUT TEKNOLOGI SEPULUH NOPEMBER SURABAYA 2026

Implementasi Algoritma Heuristik Minimum Execution Time (MET) pada Cloud Nyata Microsoft Azure

Ringkasan Eksekutif

Proyek ini menerapkan Minimum Execution Time (MET) pada cloud nyata Microsoft Azure untuk menjadwalkan pemrosesan gambar JPEG sintetis ke empat VM worker. Round Robin (RR) digunakan sebagai pembanding. Evaluasi terdiri atas run baseline 100 task dan eksperimen skalabilitas 100–500 task. Pada run baseline 4 worker, MET menyelesaikan pekerjaan dalam 11,4768 detik, sedangkan RR membutuhkan 5,4666 detik. Throughput RR mencapai 18,2928 task/detik dibandingkan 8,7132 task/detik pada MET. MET menempatkan seluruh 100 task pada vm-worker-1; RR membagi 25 task ke setiap worker. Hasil ini menunjukkan bahwa pada implementasi dan workload yang diuji, MET mengalami penumpukan task karena tidak memperhitungkan antrean atau beban VM.

Eksperimen skalabilitas terpisah menguji beban 100, 200, 300, 400, dan 500 task dari satu dataset 500 gambar sintetis, masing-masing dengan tiga pengulangan untuk MET dan tiga untuk RR (30 run, seluruhnya selesai). Makespan MET naik dari 11,744 s (100 task) menjadi 56,807 s (500 task), sedangkan RR dari 5,769 s menjadi 28,510 s; pada semua ukuran beban RR sekitar 1,97–2,04 kali lebih cepat. Throughput kedua algoritma relatif mendatar (MET 8,39–8,80 task/s; RR 16,52–17,54 task/s). DI MET tetap 4,000 karena seluruh task selalu dikirim ke vm-worker-1, sedangkan DI RR berada pada 0,565–0,654. Utilisasi berbasis waktu eksekusi rata-rata sekitar 24% untuk MET dan 41,8–43,0% untuk RR. Simpangan baku sampel antar-pengulangan kecil, tetapi hanya ada tiga pengulangan pada dataset yang sama sehingga hasil ini tidak dinyatakan signifikan secara statistik.

Hasil masih memiliki keterbatasan penting. Label load_300 pada baseline ternyata tetap memproses 100 task karena manifest awal hanya berisi 100 gambar; keterbatasan ini ditangani melalui eksperimen skalabilitas terpisah dengan dataset 500 gambar (Bagian 5.3). Uji resize tidak memiliki dokumentasi ukuran VM sebelum/sesudah. Uji failover Azure menghentikan vm-worker-4, bukan VM yang menjadi target MET. Run baseline hanya satu run per algoritma. Kesimpulan skalabilitas dibatasi pada rentang 100–500 task, empat worker, tiga pengulangan, dan satu dataset gambar sintetis; kesimpulan tentang dampak resize dan failover target tetap harus dibatasi.

Bab 1. Pendahuluan dan Arsitektur Azure

1.1 Latar Belakang dan Tujuan

Pada tugas sebelumnya, MET dievaluasi melalui simulasi CloudSim dengan 20 VM heterogen dan workload GoCJ. Pada proyek ini, MET dijalankan pada cloud nyata dengan workload pemrosesan gambar. Tujuannya adalah menguji perilaku algoritma dalam sistem terdistribusi yang benar-benar melakukan pemrosesan, serta membandingkannya dengan Round Robin menggunakan makespan, throughput, waktu tunggu, Degree of Imbalance (DI), dan utilisasi berbasis waktu sibuk.

1.2 Arsitektur

Arsitektur menggunakan satu VM master untuk menjalankan master.py dan scheduler.py, serta empat VM worker untuk memproses gambar. Repo mengelompokkan vm-worker-1 dan vm-worker-2 sebagai Host A, serta vm-worker-3 dan vm-worker-4 sebagai Host B. Pengelompokan ini adalah label arsitektur proyek; keberadaan dua host fisik Azure perlu

dibuktikan melalui konfigurasi Azure yang sebenarnya, karena Azure umumnya tidak menampilkan pemetaan server fisik pelanggan.

VM	Peran	vCPU yang teramati	Ukuran SKU Azure
vm-master	Master / scheduler	2 (nproc, pengamatan OS)	2
vm-worker-1	Worker	2 (dilaporkan /health)	2
vm-worker-2	Worker	2 (dilaporkan /health)	2
vm-worker-3	Worker	2 (dilaporkan /health)	2
vm-worker-4	Worker	2 (dilaporkan /health)	2

Catatan: jumlah vCPU berasal dari pengamatan sistem operasi (nproc/lscpu pada master) dan respons /health pada worker. Ukuran/SKU VM, RAM, region, Resource Group, konfigurasi NSG, dan biaya Azure tidak tersedia pada bukti yang dilampirkan sehingga ditandai belum terverifikasi dan perlu diisi dari portal Azure jika ingin dicantumkan.

Bab 2. Dataset dan Worker

2.1 Dataset

Dataset dibuat secara sintetis oleh dataset_generator.py dengan seed 42. Manifest berisi 100 gambar JPEG: 30 gambar berukuran 640×480, 30 berukuran 1280×720, 30 berukuran 1920×1080, dan 10 berukuran 2560×1440. Panjang task abstrak dihitung sebagai length = lebar × tinggi / 10000. Total length dataset adalah 13.593,60. Label load_300 pada hasil tidak menunjukkan 300 gambar: manifest yang tersedia tetap 100 baris, sehingga --limit 300 hanya memuat 100 task.

Dataset skalabilitas. Untuk eksperimen skalabilitas dibuat dataset terpisah berisi 500 gambar JPEG sintetis dengan dataset_generator.py (--count 500 --seed 42) di folder data_500 pada VM master: 500 file gambar dan manifest 501 baris (satu header dan 500 task). Komposisi: 150 gambar 640×480, 150 gambar 1280×720, 150 gambar 1920×1080, dan 50 gambar 2560×1440. Total ukuran gambar yang dilaporkan generator adalah 259.394.538 byte dan total length 67.968,00. Dataset dibuat satu kali dan dipakai pada seluruh run skalabilitas; beban N task adalah N task pertama pada manifest (task_id 1 sampai N), sehingga beban kecil merupakan himpunan bagian beban besar. Total length per ukuran beban adalah 14.008,32 (100), 28.454,40 (200), 42.800,64 (300), 55.795,20 (400), dan 67.968,00 (500). Dataset ini adalah workload pemrosesan gambar sintetis, bukan GoCJ, dan terpisah dari dataset baseline 100 gambar di atas.

2.2 Worker

worker.py menyediakan layanan FastAPI pada port 8000. Endpoint GET /health mengembalikan status, identitas VM, dan jumlah core. Endpoint POST /process menerima gambar dan menjalankan pipeline Pillow berupa konversi RGB, Gaussian blur, resize LANCZOS pada skala 0,5, unsharp mask, dan penyimpanan JPEG kualitas 85. Pipeline dijalankan dua kali; pass kedua bekerja pada gambar yang telah diperkecil. Worker memakai ProcessPoolExecutor berdasarkan jumlah vCPU yang dilaporkan.

Bab 3. Algoritma dan Metrik

3.1 Minimum Execution Time (MET)

MET menghitung estimasi waktu ET(i,j) = length_i / capacity_j untuk setiap task i dan VM j, lalu memilih VM dengan estimasi waktu terkecil. Implementasi memakai toleransi seri 5%; jika kapasitas berada dalam toleransi tersebut, VM yang

muncul lebih dahulu dalam daftar dipilih. Karena MET tidak memperhitungkan antrean atau beban yang sudah diberikan, task dapat terus menumpuk pada satu VM.

3.2 Round Robin

Round Robin membagikan task secara bergiliran ke worker yang tersedia. Pada baseline 100 task dan empat worker, RR membagikan 25 task ke setiap worker. Pembagian jumlah task sama tidak berarti beban total sama karena gambar memiliki ukuran dan length berbeda.

3.3 Definisi Metrik

Metrik	Definisi yang dipakai
Makespan	Waktu sampai task terakhir selesai.
Throughput	Jumlah task / makespan, satuan task/detik.
avg_wait_s	Rata-rata send_s, yaitu waktu relatif ketika task mulai dikirim dari master; bukan waktu antre murni yang diukur terpisah.
DI	`(Tmax − Tmin) / Tavg`, dengan `Tj` berupa `finish_s` terbesar pada VM; VM tanpa task diberi nilai 0.
Utilisasi rata-rata	Rata-rata util_per_vm; util_per_vm = jumlah exec_s / (cores × makespan) × 100%. Ini bukan CPU% dari sistem operasi.
Waktu respons	finish_s karena semua task dianggap tersedia pada t0.
Waktu layanan	`finish_s − send_s`.

Self-test scheduler.py dilaporkan lulus dan menghasilkan DI = 20,0 untuk model 20 VM, konsisten dengan hasil simulasi terdahulu. Ini adalah uji logika scheduler, bukan pengukuran performa Azure.

Definisi di atas diverifikasi ulang terhadap data skalabilitas (30 file tasks_*.csv). avg_wait_s sama dengan rata-rata kolom send_s (selisih maksimum 0,00005 s terhadap summary.csv). DI dapat direproduksi dari finish_s terbesar per worker dengan rumus (Tmax − Tmin) / Tavg (selisih maksimum 0,00005). Utilisasi per VM direproduksi dari jumlah exec_s / (2 × makespan) × 100% dengan 2 core per worker (selisih rata-rata maksimum 0,005 poin persen). Makespan pada summary.csv sedikit lebih besar (hingga 0,049 s) daripada finish_s terbesar pada file task; penyebab pastinya tidak diverifikasi dan diduga berasal dari pencatatan waktu pada master. Karena itu avg_wait_s adalah ukuran relatif waktu pengiriman dari master dengan asumsi semua task tersedia pada t = 0, bukan waktu antre murni, dan tidak boleh disamakan dengan finish_s − send_s maupun exec_s.

Bab 4. Master dan Alur Eksekusi

master.py membaca manifest dan konfigurasi worker, memeriksa kesehatan worker melalui /health, mengkalibrasi kapasitas, memanggil scheduler untuk menentukan tujuan task, mengirim gambar ke worker, menangani worker yang tidak sehat, menghitung metrik, lalu menulis summary.csv dan tasks_<ALGO>_<timestamp>.csv. Kalibrasi menggunakan gambar 1920×1080 dengan length 207,36. Kapasitas dihitung dari cores × 207,36 dibagi waktu dinding yang diamati master. Karena waktu dinding dapat mencakup transfer dan latensi jaringan, kapasitas ini tidak selalu sama dengan kecepatan pemrosesan CPU murni.

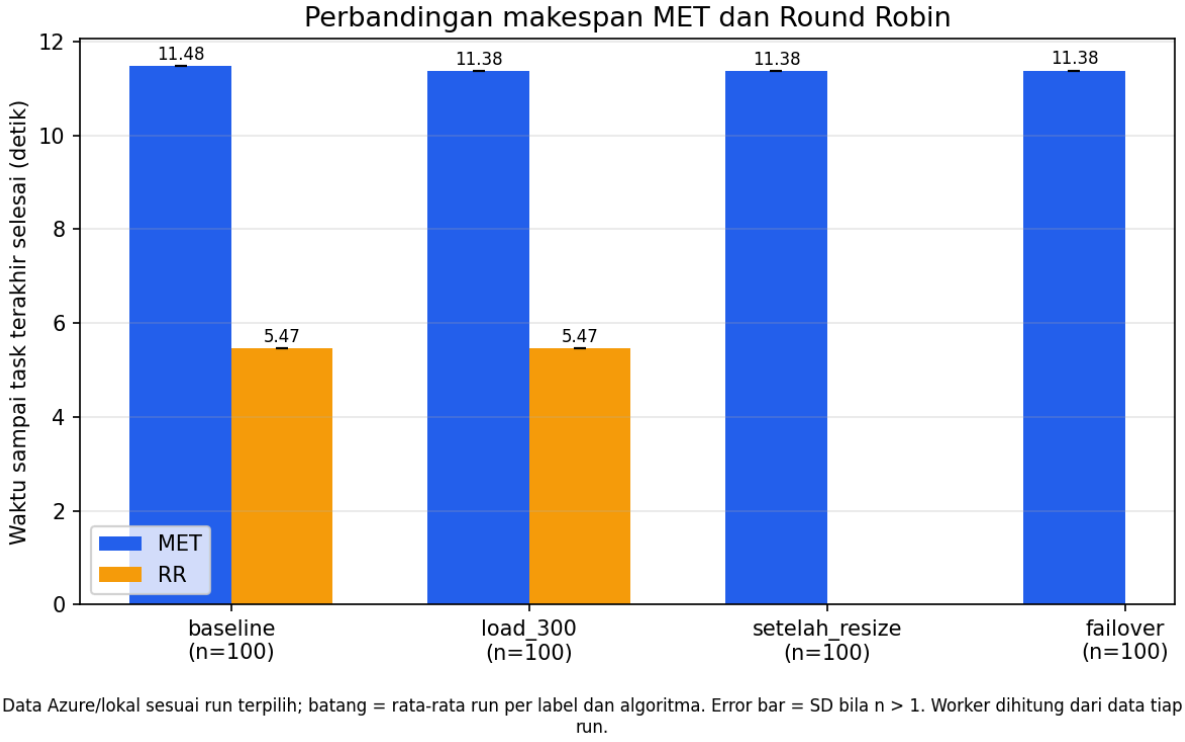
summary.csv menyimpan satu baris per run; tasks_*.csv menyimpan data per task. File tasks dipasangkan ke summary melalui timestamp. Empat run awal menggunakan tiga worker dan tidak dicampur ke perbandingan baseline empat worker.

Bab 5. Pengujian, Hasil, Analisis, dan Kesimpulan

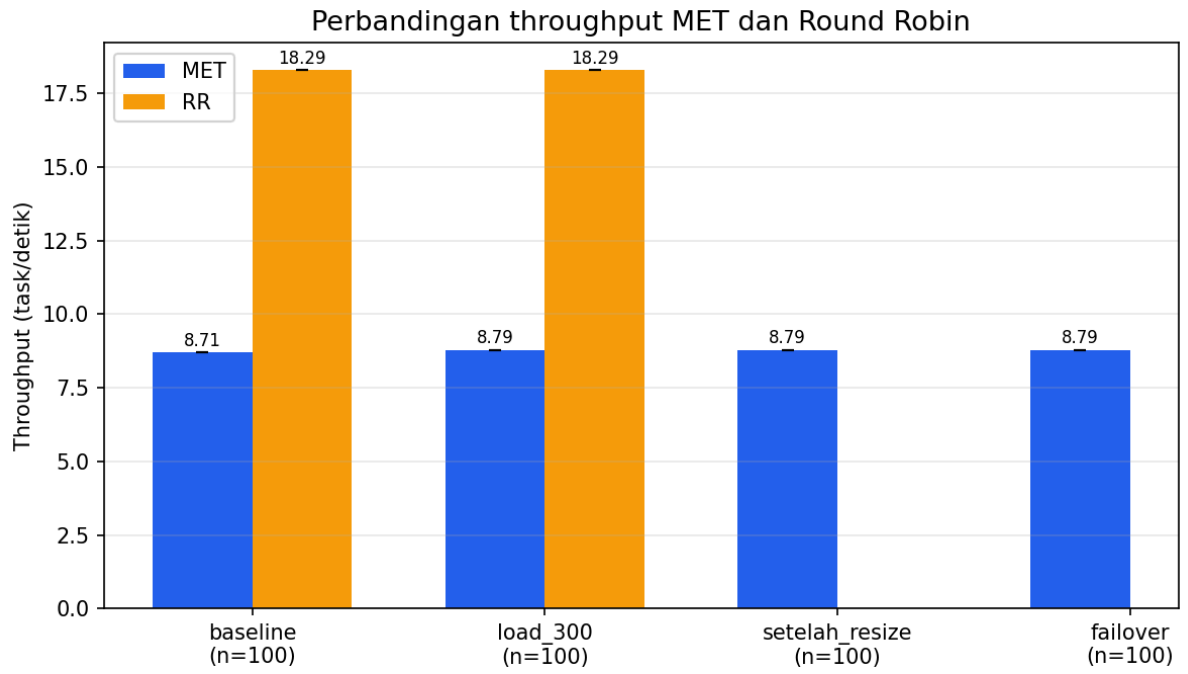
5.1 Hasil Baseline Azure (100 Task)

Metrik	MET	Round Robin	Interpretasi ringkas
Jumlah task	100	100	Beban sama.
Makespan (s)	11,4768	5,4666	RR sekitar 2,10 kali lebih cepat.
Waktu tunggu rata-rata (s)	5,5845	1,9266	Nilai avg_wait_s MET sekitar 2,90 kali RR.
Throughput (task/s)	8,7132	18,2928	RR sekitar 2,10 kali throughput MET.
DI	4,0000	0,6042	MET mencapai kondisi timpang ekstrem untuk empat VM.
Utilisasi rata-rata (%)	23,78	42,51	MET hanya memakai satu dari empat worker.
Respons rata-rata (s)	5,8124	2,2599	Dihitung dari finish_s task CSV.
Respons median (s)	5,8390	2,0868	Dihitung dari finish_s task CSV.
Waktu layanan rata-rata (s)	0,2278	0,3334	`finish_s − send_s`; definisi operasional sesuai handoff.

Sumber: run baseline empat worker dengan timestamp 2026-10-29 02:03:04 untuk MET dan 2026-10-29 02:03:11 untuk RR. Tiap algoritma hanya memiliki satu run baseline yang dipakai di tabel; tidak ada dasar untuk uji signifikansi statistik.

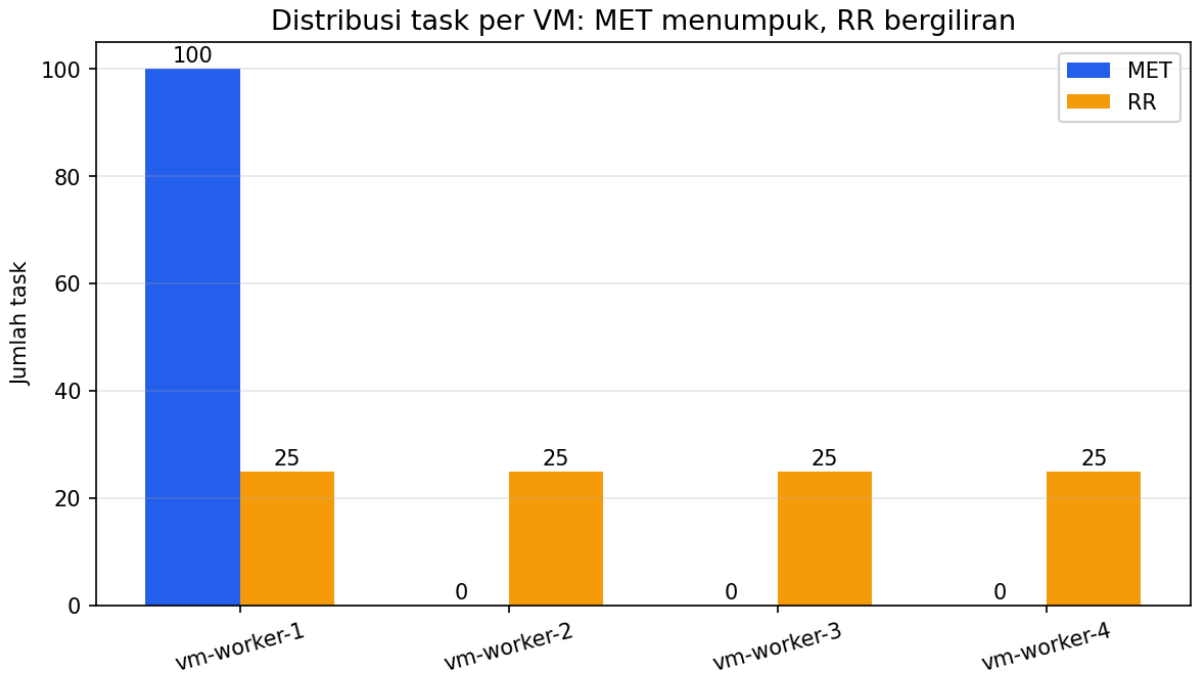


Gambar 5.1. Perbandingan makespan MET dan Round Robin.



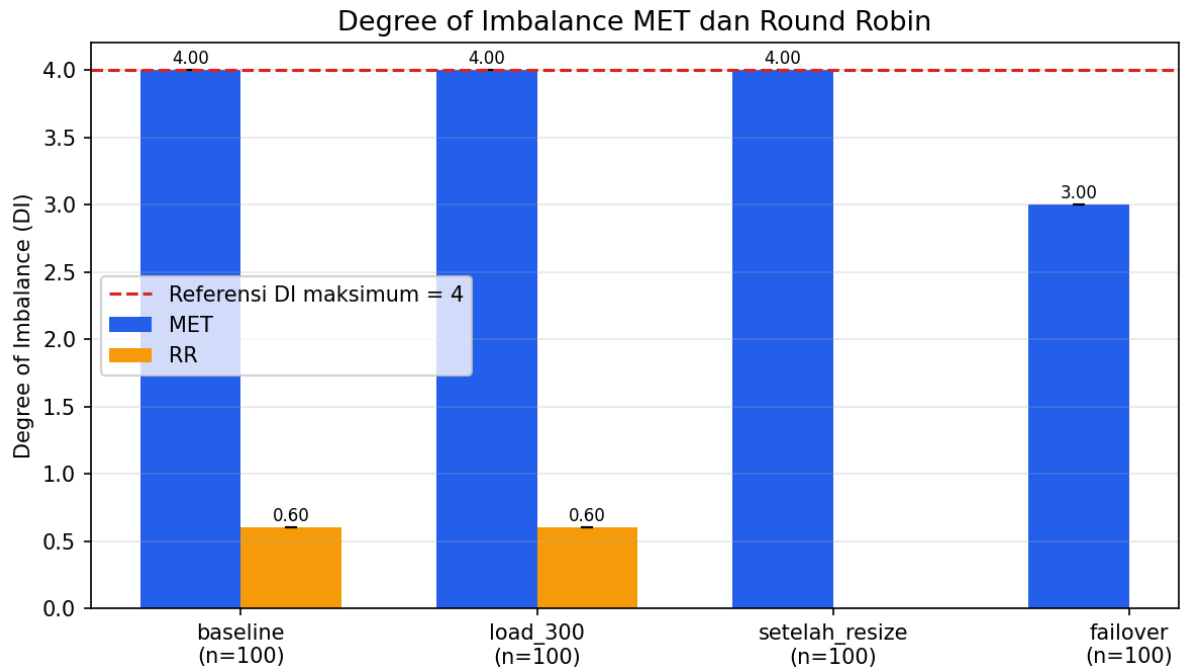
Data Azure/lokal sesuai run terpilih; batang = rata-rata run per label dan algoritma. Error bar = SD bila n > 1. Worker dihitung dari data tiap run.

Gambar 5.2. Perbandingan throughput MET dan Round Robin.



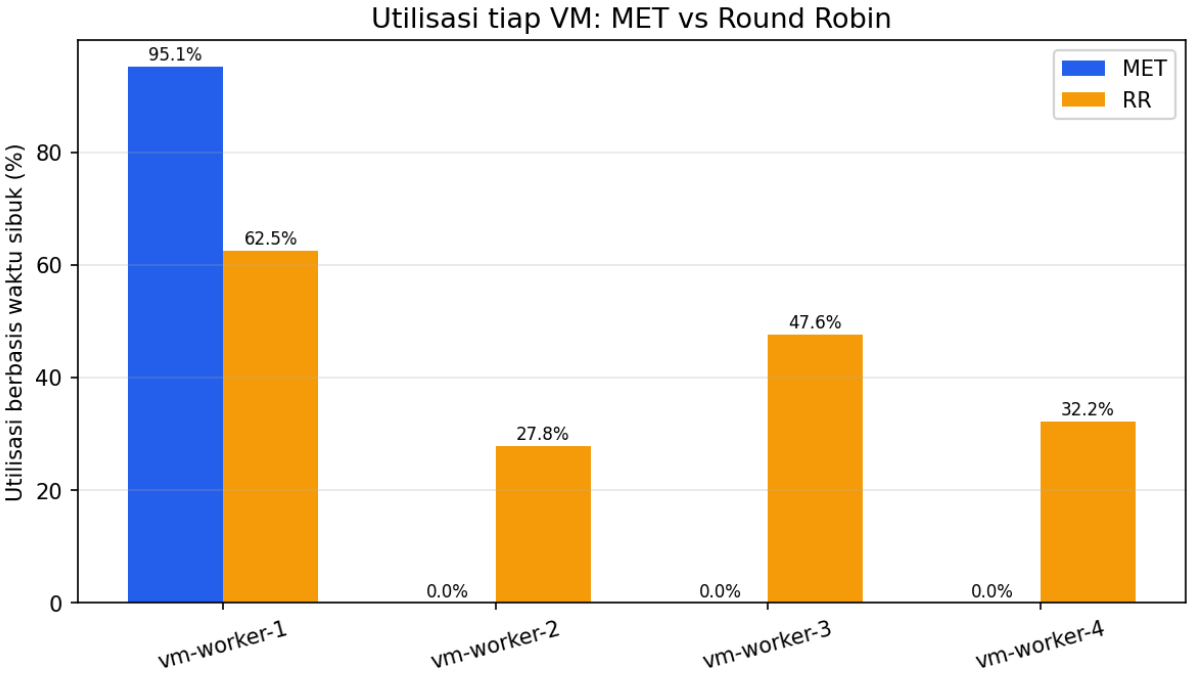
Baseline terakhir per algoritma; MET n=100, RR n=100. Jumlah task bukan ukuran total beban piksel.

Gambar 5.3. Distribusi task pada setiap VM.



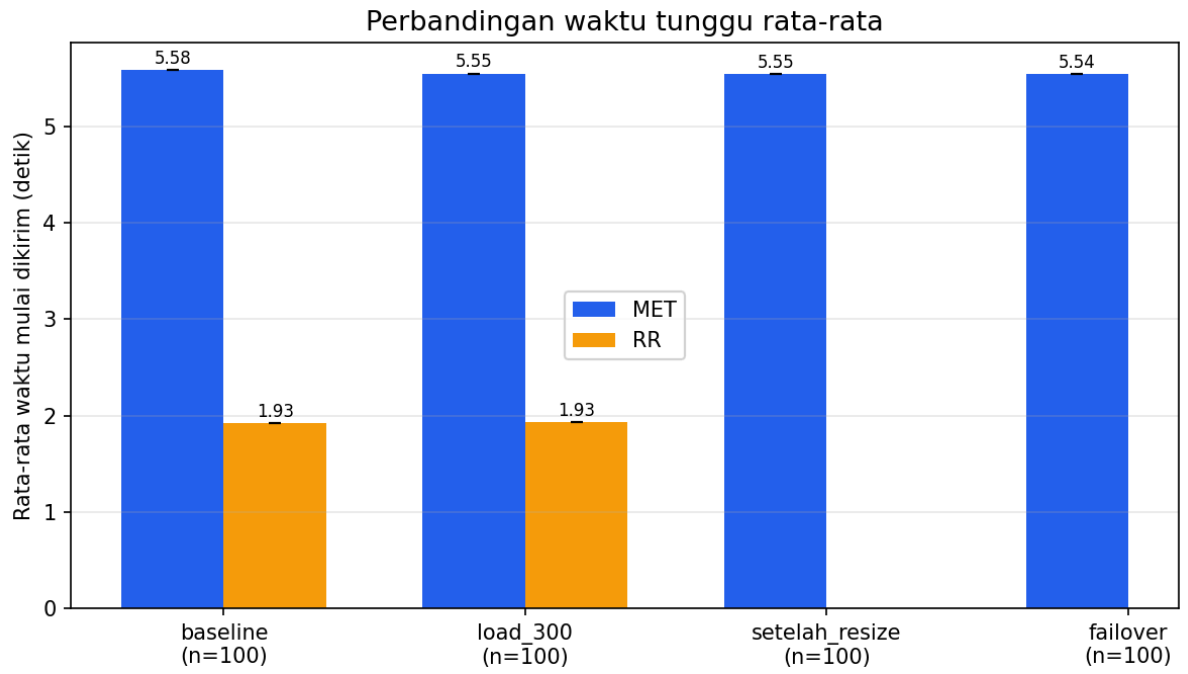
Data Azure/lokal sesuai run terpilih; batang = rata-rata run per label dan algoritma. Error bar = SD bila n > 1. Worker dihitung dari data tiap run.

Gambar 5.4. Degree of Imbalance (DI).



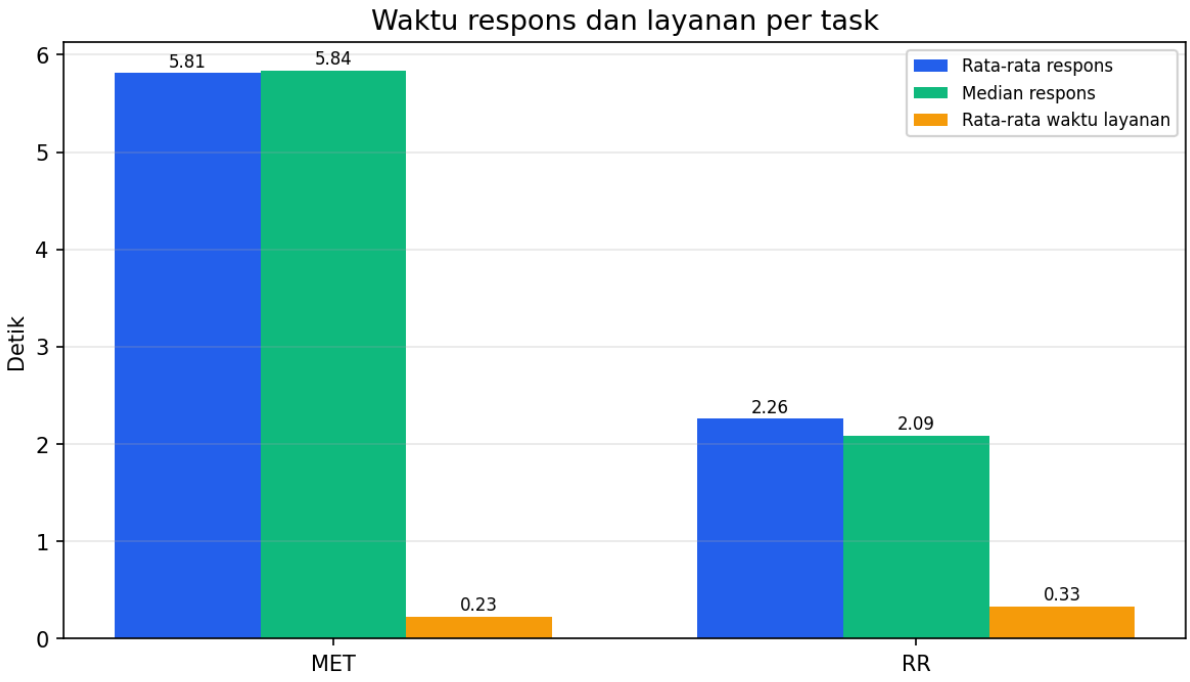
Utilisasi dihitung dari jumlah exec_s dibagi cores × makespan, bukan CPU% dari Azure Monitor.

Gambar 5.5. Utilisasi berbasis waktu sibuk per VM.



Data Azure/lokal sesuai run terpilih; batang = rata-rata run per label dan algoritma. Error bar = SD bila n > 1. Worker dihitung dari data tiap run.

Gambar 5.6. Perbandingan avg_wait_s.



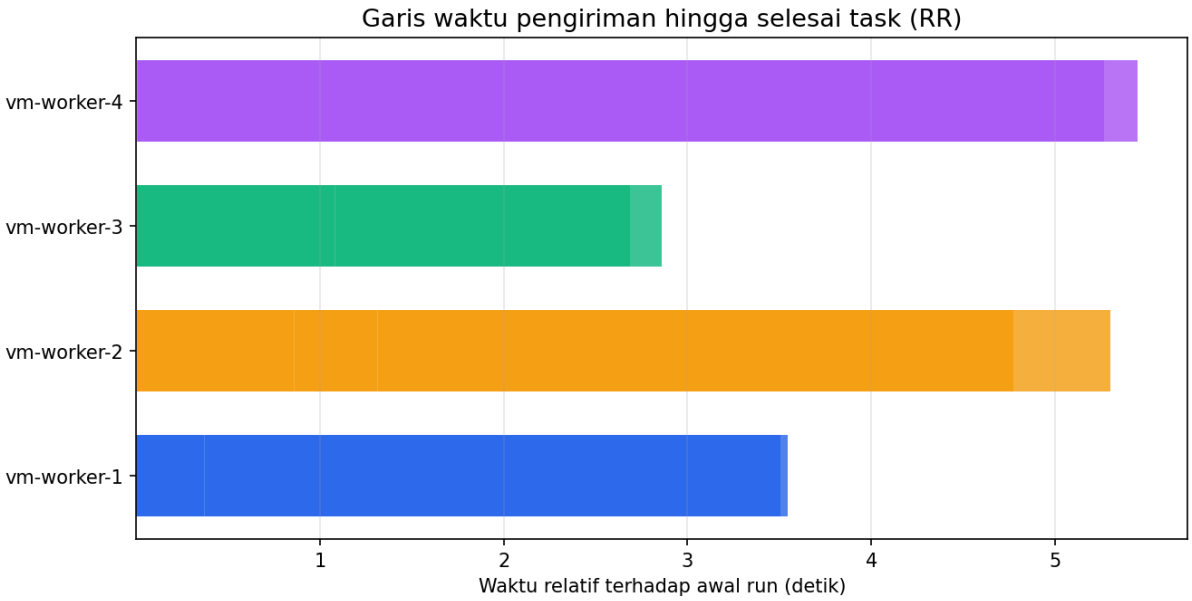
Respons = finish_s karena semua task tersedia pada t0; layanan = finish_s − send_s sesuai definisi data proyek.

Gambar 5.7. Respons dan waktu layanan per task.



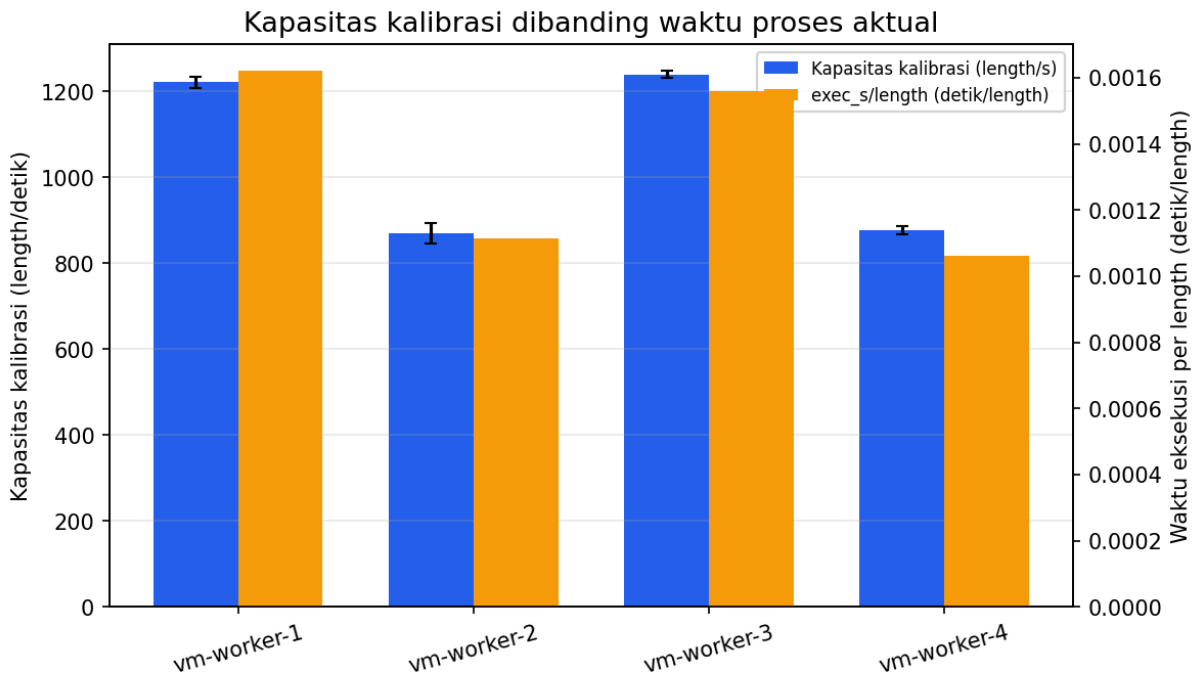
Baseline terakhir, n=100 task; batang membentang dari send_s sampai finish_s dan mencakup antrean/pengiriman.

Gambar 5.8. Garis waktu task pada run MET baseline.



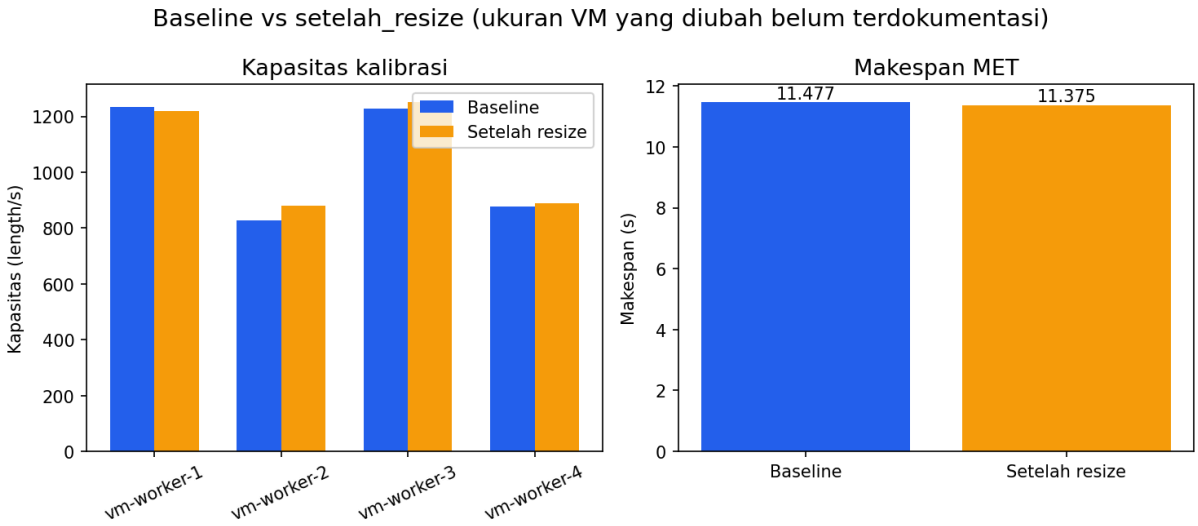
Baseline terakhir, n=100 task; batang membentang dari send_s sampai finish_s dan mencakup antrean/pengiriman.

Gambar 5.9. Garis waktu task pada run RR baseline.



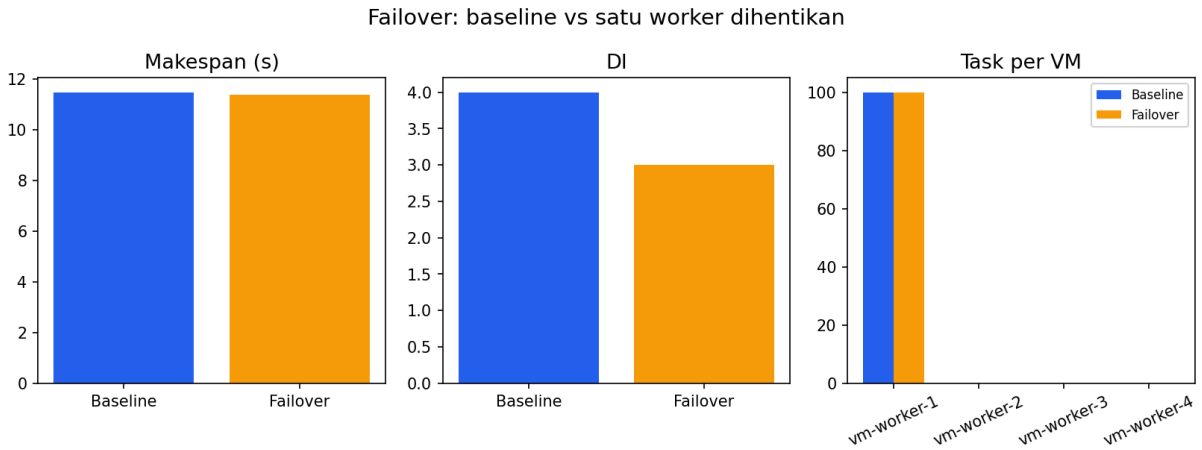
Kapasitas kalibrasi dipengaruhi waktu dinding dari master; exec_s/length dihitung dari log RR baseline terakhir yang tersedia.

Gambar 5.10. Kapasitas kalibrasi dibanding exec_s per length.



VM tujuan MET baseline dan setelah resize: vm-worker-1 dan vm-worker-1; hasil tidak membuktikan efek resize tanpa catatan ukuran VM.

Gambar 5.11. Baseline dibanding label setelah_resize; dampak resize belum terverifikasi.



Gambar 5.12. Baseline dan failover saat worker non-target MET dihentikan.

5.2 Analisis Baseline

5.2.1 MET dibanding Round Robin

Pada baseline 100 task, RR menyelesaikan pekerjaan dalam 5,4666 detik, dibanding 11,4768 detik pada MET. Throughput RR 18,2928 task/detik, sedangkan MET 8,7132 task/detik. Log distribusi menunjukkan MET mengirim 100 task ke vm-worker-1, sedangkan RR membagi 25 task per worker. Penjelasan yang konsisten dengan kode adalah MET mengabaikan antrean dan beban yang sudah terjadwal, sementara RR menggunakan semua worker secara paralel.

5.2.2 Ketimpangan dan Utilisasi

DI MET = 4,0 sama dengan jumlah worker pada kondisi seluruh task menumpuk di satu worker. Utilisasi vm-worker-1 tercatat 95,12% dan tiga worker lainnya 0%; rata-rata empat worker menjadi 23,78%. Pada RR, DI = 0,6042 dan utilisasi rata-rata 42,51%. RR tidak mencapai utilisasi 100% karena beban length tidak identik, performa worker berbeda, dan worker yang selesai lebih awal menunggu worker lain menyelesaikan task. Transfer gambar dan waktu dinding juga memengaruhi durasi run, tetapi utilisasi proyek dihitung dari exec_s saja.

5.2.3 Kalibrasi dan Kapasitas

Kapasitas kalibrasi baseline MET tercatat sekitar 1235,49 length/detik untuk vm-worker-1 dan 1229,47 length/detik untuk vm-worker-3. Selisihnya kurang dari toleransi 5%, sehingga tie-break memilih vm-worker-1 karena urutannya lebih awal. Log RR menunjukkan total exec_s per length tidak sepenuhnya mengikuti urutan kapasitas kalibrasi. Salah satu kemungkinan adalah pengukuran kapasitas dari master ikut memasukkan transfer/latensi jaringan; ini merupakan dugaan yang perlu diuji dengan kalibrasi berbasis waktu pemrosesan worker dan pengukuran jaringan terpisah.

5.2.4 Beban 300 Task

Run berlabel load_300 mencatat n_tasks = 100, sama dengan baseline. Oleh karena itu, hasil ini tidak dapat digunakan untuk menyimpulkan throughput saat beban dinaikkan menjadi 300 task, dan grafik skalabilitas tidak dibuat dari data tersebut. Keterbatasan ini ditangani dengan membuat dataset 500 gambar di folder terpisah dan menjalankan eksperimen skalabilitas dengan n_tasks yang terverifikasi pada summary.csv; hasilnya dibahas pada Bagian 5.3 dan tidak dicampur dengan baseline ini.

5.2.5 Resize dan Failover

Run setelah_resize mencatat makespan MET 11,3753 detik dan throughput 8,7910 task/detik, namun ukuran VM sebelum/sesudah tidak tercatat dan kapasitas hasil kalibrasi hampir tidak berubah. Dampak resize belum dapat disimpulkan. Pada run failover, vm-worker-4 dihentikan; MET tetap menaruh seluruh task di vm-worker-1, makespan 11,3772 detik dan DI 3,0 karena hanya tiga worker sehat. Ini menguji hilangnya worker non-target, bukan skenario paling kritis saat vm-worker-1 sebagai target MET dihentikan. Uji failover lokal terhadap target MET memiliki bukti di repo, tetapi tidak menggantikan uji target di Azure.

5.3 Eksperimen Skalabilitas Azure (100–500 Task)

5.3.1 Metode Eksperimen

Eksperimen memakai satu master dan empat worker (vm-worker-1 sampai vm-worker-4) dengan dataset 500 gambar pada Bagian 2.1. Jumlah task yang diuji adalah 100, 200, 300, 400, dan 500. Setiap ukuran beban dijalankan tiga kali (rep1–rep3) untuk MET dan tiga kali untuk RR, sehingga total 5 ukuran beban × 3 pengulangan × 2 algoritma = 30 run. Setiap run diberi label scale_<n>_rep<k> pada summary.csv (misalnya scale_100_rep1) sehingga dapat dibedakan dari baseline. Log mencatat SEMUA PENGUJIAN SELESAI. dan seluruh 30 run selesai. Smoke test 20 task yang dijalankan sebelum eksperimen utama hanya untuk memeriksa sistem dan tidak dimasukkan ke agregat maupun grafik.

Validasi data sebelum analisis: summary.csv berisi 30 baris data (31 baris dengan header); setiap kombinasi jumlah task dan algoritma memiliki tepat tiga label rep1–rep3 tanpa duplikat; n_tasks sesuai angka pada label; terdapat 30 file tasks_*.csv dan jumlah barisnya sama dengan n_tasks tiap run; jumlah task per worker pada kolom tasks_per_vm berjumlah n_tasks; pemeriksaan kesehatan menunjukkan 4 dari 4 worker sehat pada seluruh run; dan run.log tidak memuat pesan galat. Satu-satunya peringatan adalah peringatan master bahwa estimasi makespan berada di luar rentang 30–180 detik, yang muncul pada beban 100 dan 200 task (estimasi sekitar 11 dan 23 detik) tetapi tidak pada 300–500 task. Tabel agregat scalability_mean_sd.csv (50 baris data) dihitung ulang dari summary.csv dan hasilnya identik.

Agregasi: untuk setiap kombinasi metrik, jumlah task, dan algoritma dihitung mean dan simpangan baku sampel (penyebut n − 1) dari tiga pengulangan. Batang galat pada grafik adalah mean ± 1 sample SD. Tiga pengulangan hanya menangkap variasi eksekusi (runtime) pada dataset dan seed yang sama; ini bukan variasi antar-dataset dan bukan dasar untuk menyatakan perbedaan signifikan secara statistik. Waktu kalibrasi kapasitas dijalankan ulang pada setiap run, sehingga kapasitas kalibrasi berbeda sedikit antar-run (mis. vm-worker-1 1.201,48–1.239,96 length/s dan vm-worker-4 773,87–871,87 length/s).

Urutan eksekusi yang tercatat pada timestamp summary.csv adalah beban menaik 100→500 dalam setiap pengulangan, dengan MET dijalankan sebelum RR pada setiap ukuran beban. Urutan ini sama pada ketiga pengulangan; efek urutan tidak diuji dan dicantumkan sebagai keterbatasan.

5.3.2 Ringkasan Hasil (mean ± sample SD)

Tabel 5.1 dan Tabel 5.2 merangkum kelima metrik untuk MET dan RR (n = 3 pengulangan per sel). Satuan: makespan dan avg_wait_s dalam detik, throughput dalam task/detik, DI tanpa satuan, utilisasi dalam persen.

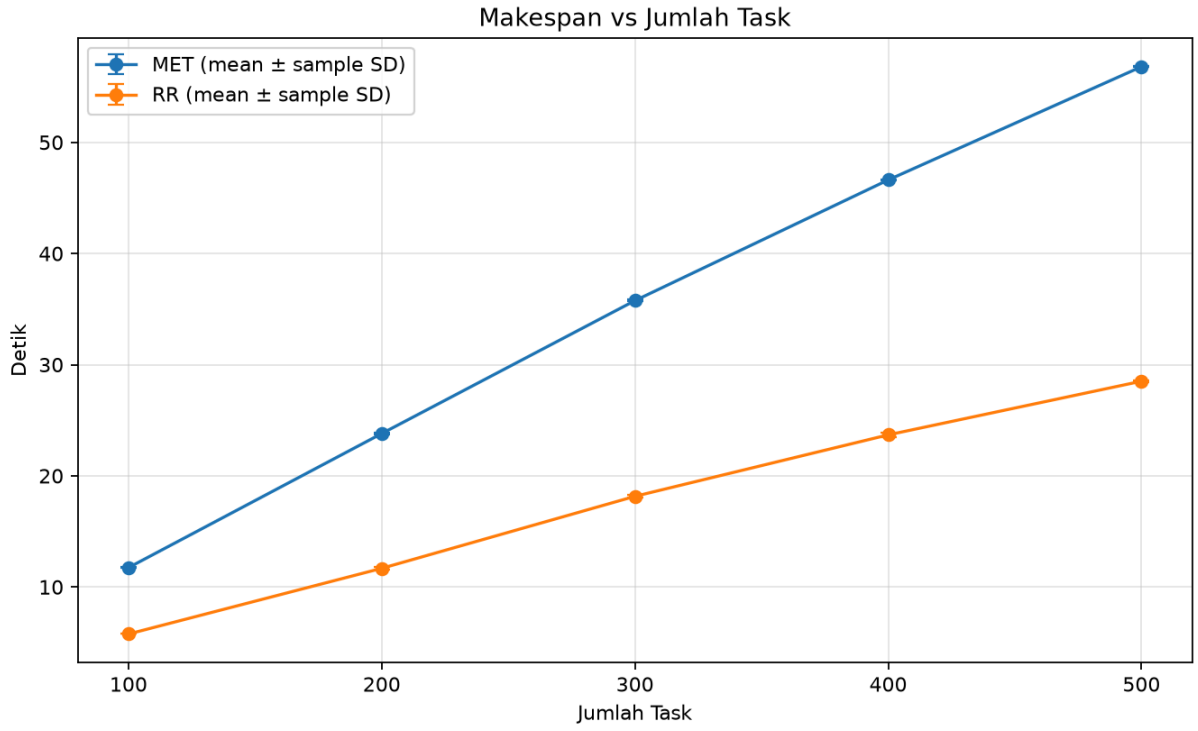
Tabel 5.1. Hasil skalabilitas MET pada Azure, mean ± sample SD dari 3 pengulangan.

Jumlah task	Makespan (s)	avg_wait_s (s)	Throughput (task/s)	DI	Utilisasi (%)
100	11,744 ± 0,029	5,975 ± 0,018	8,515 ± 0,021	4,000 ± 0,000	23,93 ± 0,10
200	23,804 ± 0,048	11,984 ± 0,015	8,402 ± 0,017	4,000 ± 0,000	23,97 ± 0,01
300	35,764 ± 0,076	17,702 ± 0,033	8,388 ± 0,018	4,000 ± 0,000	24,00 ± 0,02
400	46,620 ± 0,028	23,520 ± 0,018	8,580 ± 0,005	4,000 ± 0,000	24,06 ± 0,03
500	56,807 ± 0,029	29,109 ± 0,015	8,802 ± 0,005	4,000 ± 0,000	24,01 ± 0,01

Tabel 5.2. Hasil skalabilitas RR pada Azure, mean ± sample SD dari 3 pengulangan.

Jumlah task	Makespan (s)	avg_wait_s (s)	Throughput (task/s)	DI	Utilisasi (%)
100	5,769 ± 0,001	2,030 ± 0,009	17,335 ± 0,004	0,565 ± 0,002	42,64 ± 0,32
200	11,678 ± 0,100	4,262 ± 0,025	17,127 ± 0,147	0,573 ± 0,006	43,05 ± 0,32
300	18,157 ± 0,104	6,425 ± 0,022	16,523 ± 0,095	0,645 ± 0,004	41,93 ± 0,23
400	23,685 ± 0,176	8,597 ± 0,033	16,889 ± 0,126	0,649 ± 0,007	41,78 ± 0,18
500	28,510 ± 0,100	10,697 ± 0,013	17,538 ± 0,062	0,654 ± 0,004	41,87 ± 0,05

5.3.3 Makespan vs Jumlah Task



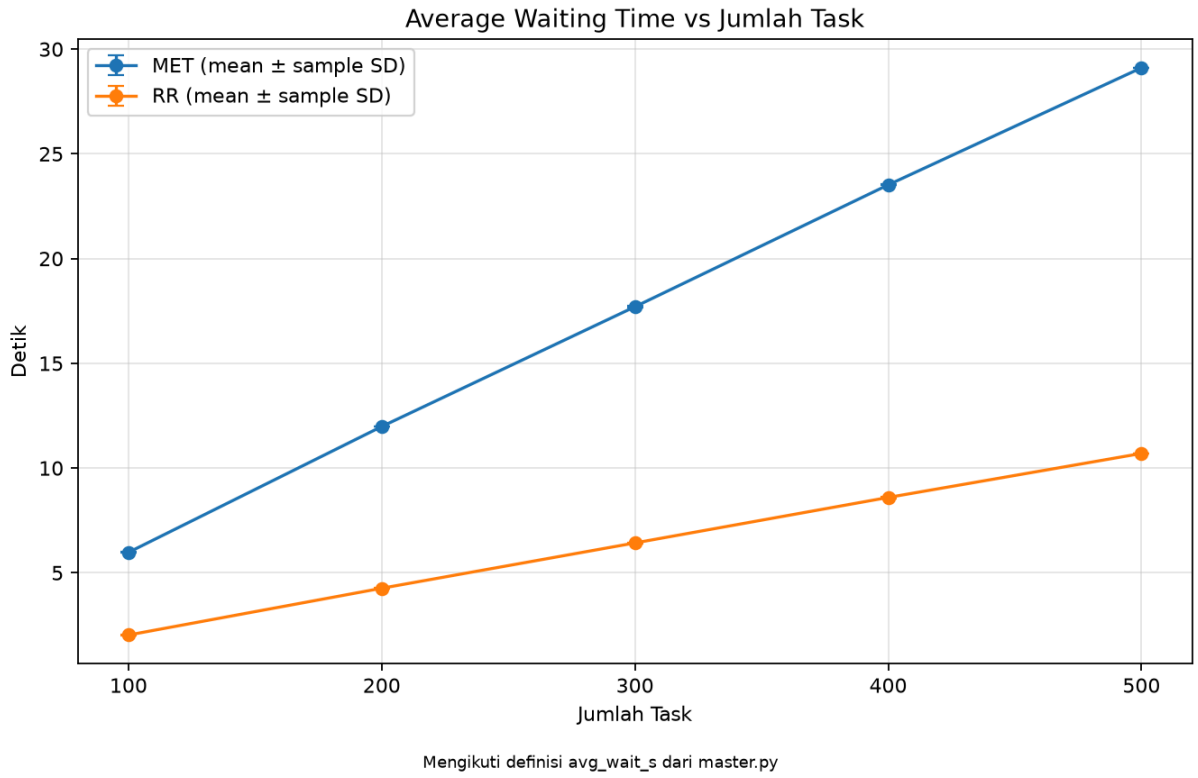
Gambar 5.13. Makespan MET dan RR terhadap jumlah task (mean ± sample SD, 3 pengulangan; batang galat sangat kecil).

Yang terlihat pada data. Makespan naik hampir lurus pada kedua algoritma. MET naik dari 11,744 s menjadi 56,807 s (sekitar 4,84 kali) dan RR dari 5,769 s menjadi 28,510 s (sekitar 4,94 kali) ketika jumlah task naik 5 kali. Total length naik 4,85 kali (14.008,32 menjadi 67.968,00), sehingga kenaikan makespan mengikuti total beban dan kira-kira sebanding. Waktu per task relatif stabil: MET 0,114–0,119 s/task dan RR 0,057–0,061 s/task. Rasio makespan MET terhadap RR adalah 2,04; 2,04; 1,97; 1,97; dan 1,99 untuk 100–500 task, yaitu RR sekitar dua kali lebih cepat pada seluruh ukuran beban yang diuji.

Interpretasi. Laju efektif MET (total length / makespan) pada 500 task adalah sekitar 1.196 length/s, setara dengan kapasitas kalibrasi satu worker (sekitar 1.200–1.240 length/s), sedangkan RR sekitar 2.384 length/s, sekitar dua kali lipat. Ini konsisten dengan pola distribusi: MET mengirim seluruh task ke satu worker sehingga hanya satu worker yang bekerja, sementara RR membagi task ke empat worker. RR tidak mencapai kecepatan empat kali lipat; salah satu dugaan adalah

dua worker yang lebih lambat (vm-worker-2 dan vm-worker-4) menjadi penentu makespan (lihat Bagian 5.3.6). Dugaan ini belum diuji secara terpisah.

5.3.4 Average Waiting Time (avg_wait_s) vs Jumlah Task

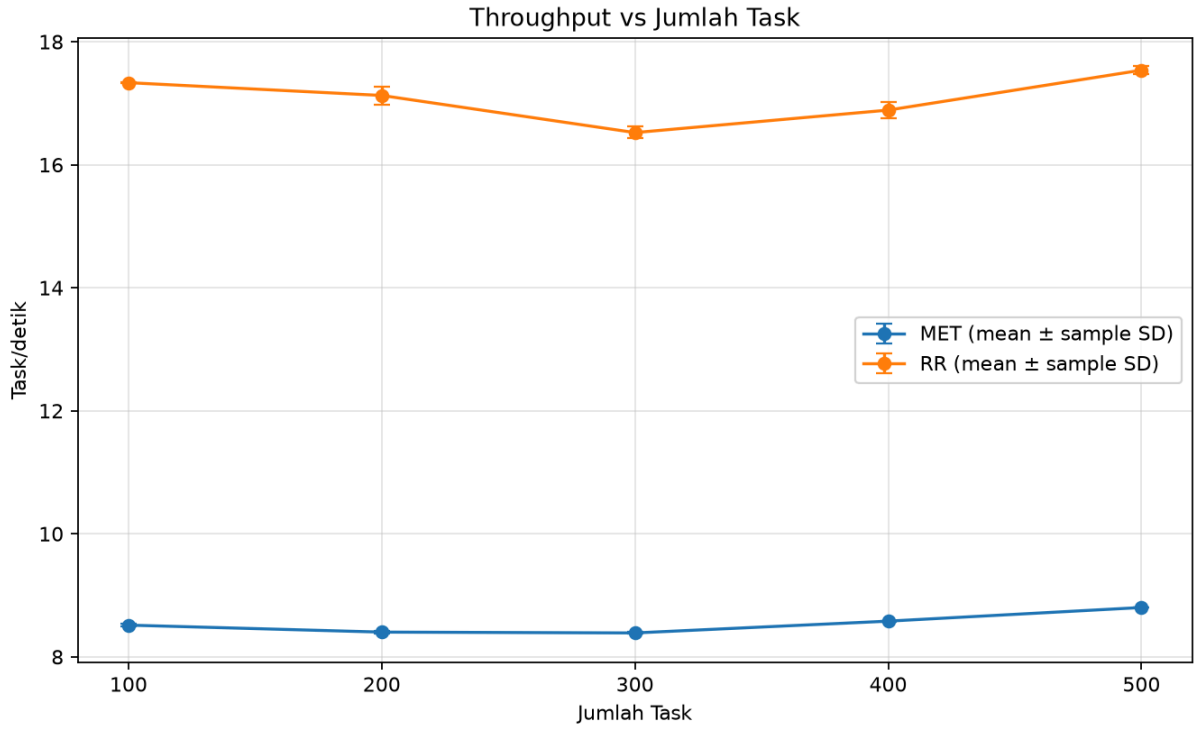


Gambar 5.14. avg_wait_s MET dan RR terhadap jumlah task (mean ± sample SD, 3 pengulangan). avg_wait_s mengikuti definisi pada master.py, yaitu rata-rata send_s.

Yang terlihat pada data. avg_wait_s naik hampir linear dengan jumlah task: MET dari 5,975 s menjadi 29,109 s dan RR dari 2,030 s menjadi 10,697 s. Nilai MET sekitar 2,7–2,9 kali nilai RR. Dibanding makespan, avg_wait_s MET sekitar 0,50–0,51 dari makespan dan RR sekitar 0,35–0,38 dari makespan.

Interpretasi. Metrik ini adalah rata-rata send_s, yaitu waktu relatif ketika task mulai dikirim oleh master dengan asumsi semua task tersedia pada t = 0; ini bukan waktu antre murni dan tidak sama dengan finish_s − send_s. Pada file task MET, send_s task terakhir (sekitar 56 s pada 500 task) hampir sama dengan makespan, sehingga pengiriman ke satu worker tersebar sepanjang run dan rata-ratanya mendekati setengah makespan. Pada RR pengiriman ke empat worker selesai lebih cepat sehingga nilainya lebih kecil. Karena pola pengiriman tidak diukur terpisah dari kapasitas penerimaan worker, penyebab rinci (pembatasan konkurensi pengirim, kecepatan worker, atau jaringan) belum dapat dipastikan; yang dapat dinyatakan hanya bahwa nilai ini meningkat seiring beban dan lebih besar pada MET.

5.3.5 Throughput vs Jumlah Task

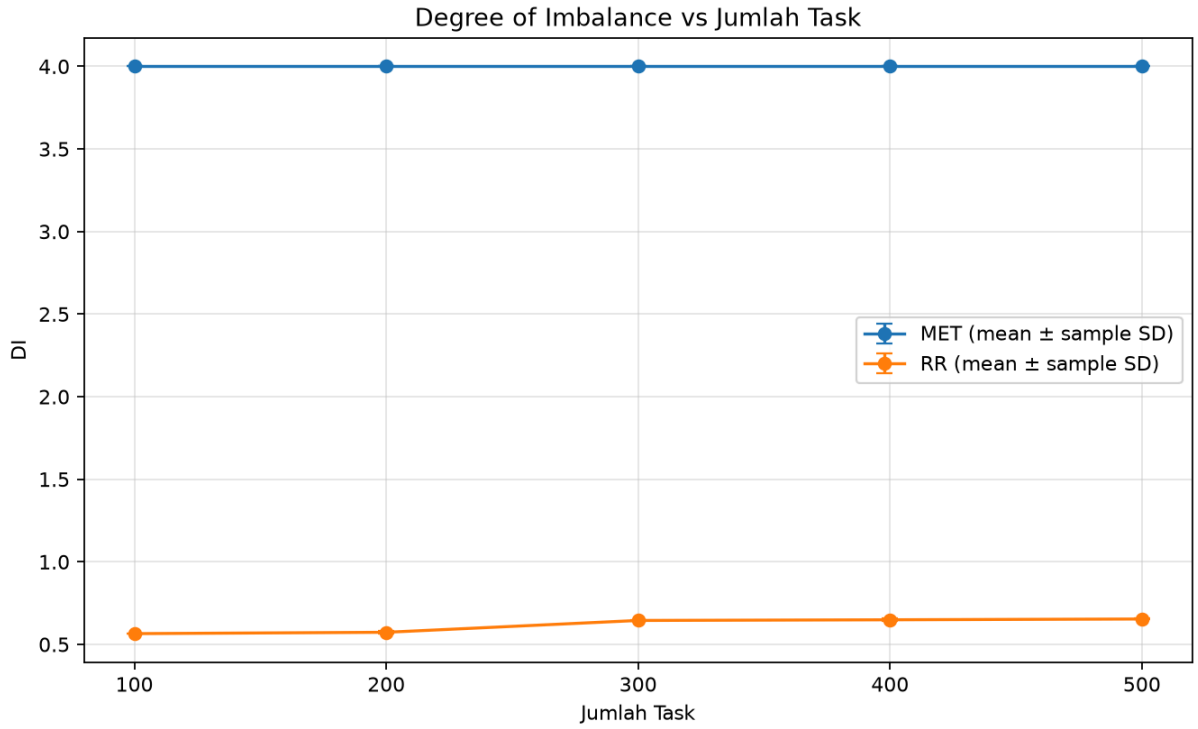


Gambar 5.15. Throughput MET dan RR terhadap jumlah task (mean ± sample SD, 3 pengulangan).

Yang terlihat pada data. Throughput tidak naik secara berarti dengan bertambahnya task; kurvanya mendatar dengan sedikit naik-turun. MET: 8,515; 8,402; 8,388; 8,580; 8,802 task/s untuk 100–500 task. RR: 17,335; 17,127; 16,523; 16,889; 17,538 task/s. Kedua kurva menurun sedikit sampai 300 task lalu naik lagi hingga 500 task, sehingga tidak monoton. RR sekitar 1,97–2,04 kali MET pada setiap ukuran beban.

Interpretasi. Throughput mendatar sesuai dengan sistem yang jenuh: begitu beberapa task masuk, pekerja yang digunakan sudah sibuk sehingga penambahan task hanya memperpanjang makespan. Pola naik-turun kecil konsisten dengan komposisi beban: rata-rata length per task pada subset 100–500 task adalah 140,08; 142,27; 142,67; 139,49; dan 135,94. Jika laju pemrosesan dianggap tetap (sekitar 1.200 length/s untuk MET), throughput yang diharapkan 8,57; 8,43; 8,41; 8,60; dan 8,83 task/s, yang mengikuti pola kurva MET. Jadi sebagian besar naik-turun kemungkinan berasal dari perbedaan komposisi subset, bukan dari efek jumlah task. Perhitungan ini merupakan pemeriksaan kecocokan sederhana, bukan pembuktian. Overhead pengiriman dan orkestrasi tidak diisolasi pada eksperimen ini.

5.3.6 Degree of Imbalance (DI) vs Jumlah Task

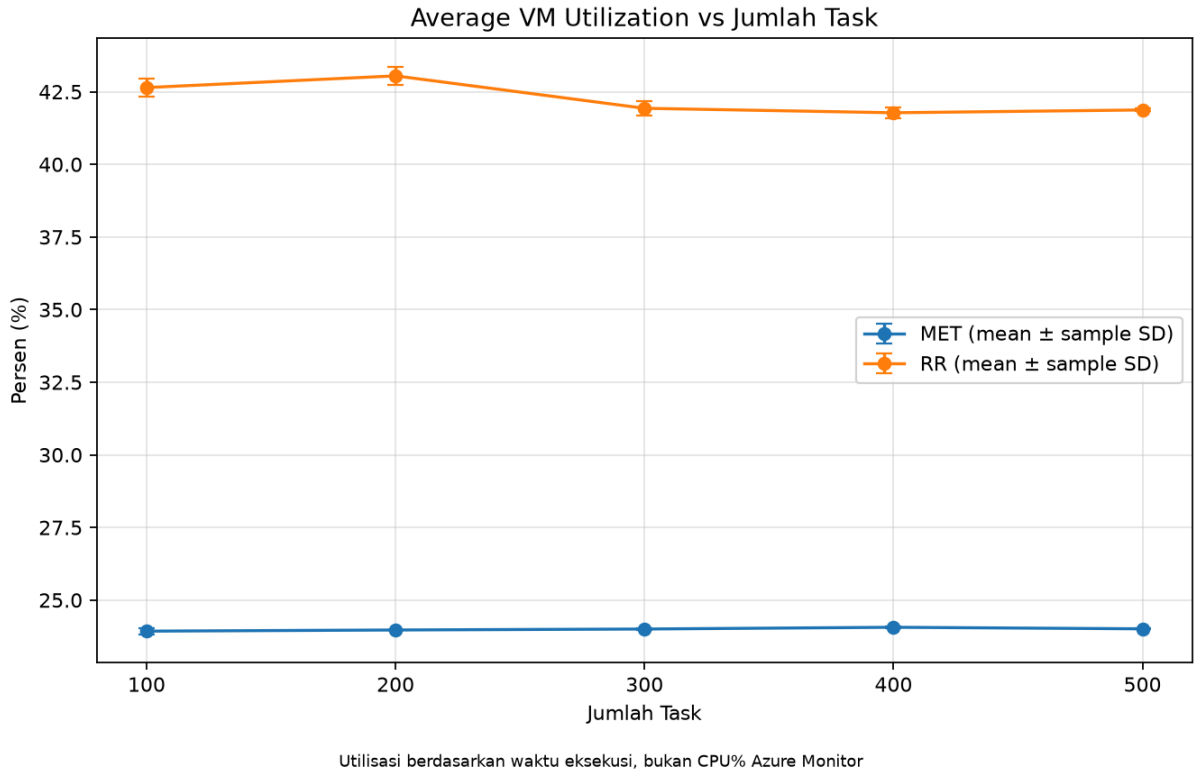


Gambar 5.16. DI MET dan RR terhadap jumlah task (mean ± sample SD, 3 pengulangan).

Yang terlihat pada data. DI MET tepat 4,000 pada semua ukuran beban dan semua pengulangan (SD = 0). DI RR jauh lebih rendah dan naik pelan: 0,565; 0,573; 0,645; 0,649; 0,654 untuk 100–500 task. Loncatan terbesar terjadi dari 200 ke 300 task, setelah itu relatif mendatar.

Interpretasi. Pada seluruh 15 run MET, seluruh task dikirim ke vm-worker-1 (100/200/300/400/500 task pada satu worker, nol pada tiga lainnya). Dengan Tj = (T, 0, 0, 0), DI = (T − 0) / (T/4) = 4, yaitu jumlah worker; nilai ini tidak bergantung pada beban. Pada 6 dari 15 run MET kapasitas kalibrasi vm-worker-3 tercatat sedikit lebih tinggi daripada vm-worker-1, tetapi selisihnya di bawah toleransi seri 5% sehingga vm-worker-1 tetap dipilih karena urutannya lebih awal. Pada RR, setiap worker menerima jumlah task yang sama, tetapi waktu selesai berbeda: pada 500 task rata-rata finish_s terakhir adalah sekitar 14,5 s (vm-worker-1), 26,8 s (vm-worker-2), 15,6 s (vm-worker-3), dan 28,5 s (vm-worker-4). Selisih ini membuat DI RR sekitar 0,65. Pada RR, jumlah exec_s total vm-worker-2 dan vm-worker-4 tidak lebih besar daripada vm-worker-1 dan vm-worker-3, namun mereka selesai jauh lebih lambat. Dugaan yang konsisten adalah adanya waktu di luar exec_s (transfer, latensi jaringan, atau antrean) yang lebih besar untuk kedua worker tersebut; waktu dinding kalibrasi dua worker ini juga lebih lama. Dugaan ini belum diuji dengan pengukuran jaringan.

5.3.7 Average VM Utilization vs Jumlah Task



Gambar 5.17. Utilisasi rata-rata MET dan RR terhadap jumlah task (mean ± sample SD, 3 pengulangan). Utilisasi berbasis waktu eksekusi, bukan CPU% Azure Monitor.

Yang terlihat pada data. Utilisasi MET hampir konstan sekitar 24% (23,93; 23,97; 24,00; 24,06; 24,01 %). Utilisasi RR lebih tinggi, yaitu 42,64; 43,05; 41,93; 41,78; 41,87 %, dengan sedikit penurunan dari 200 ke 300 task lalu mendatar.

Interpretasi. Utilisasi dihitung dari jumlah exec_s per worker dibagi (2 core × makespan), lalu dirata-rata pada empat worker; ini estimasi berbasis waktu eksekusi, bukan CPU% dari Azure Monitor atau sistem operasi. Pada MET, vm-worker-1 tercatat sekitar 96% dan tiga worker lain 0%, sehingga rata-rata empat worker sekitar 24%, yaitu seperempat dari utilisasi satu worker yang hampir penuh. Pada RR, keempat worker bekerja, tetapi utilisasi per worker berbeda: sekitar 49–55% (vm-worker-1), 27–31% (vm-worker-2), 49–54% (vm-worker-3), dan 35–38% (vm-worker-4), sehingga rata-rata sekitar 42%. RR tidak mendekati 100% karena worker yang selesai lebih awal menganggur sementara worker lain masih bekerja, dan makespan mencakup waktu di luar exec_s. Dengan demikian utilisasi RR yang lebih tinggi berasal dari pemanfaatan lebih banyak worker, bukan dari efisiensi tiap worker yang lebih baik.

5.3.8 Variasi Antar-pengulangan

Simpangan baku sampel antar-pengulangan kecil dibanding nilai mean sehingga batang galat pada grafik hampir tidak terlihat. Pada makespan, SD terbesar adalah 0,076 s untuk MET (pada 300 task, sekitar 0,2% dari mean) dan 0,176 s untuk RR (pada 400 task, sekitar 0,7% dari mean). RR cenderung memiliki variasi relatif lebih besar daripada MET; throughput RR mencapai SD 0,147 task/s dan utilisasi RR mencapai SD 0,32 poin persen. DI MET tidak bervariasi (SD = 0) dan SD DI RR paling besar 0,007. Variasi kecil ini hanya mencerminkan variasi eksekusi pada dataset yang sama dan jumlah pengulangan yang kecil (n = 3); SD dari tiga nilai sendiri tidak stabil. Perbedaan MET dan RR jauh lebih besar daripada SD, tetapi laporan ini tidak melakukan uji statistik dan tidak menyatakan signifikansi statistik.

Ringkasan temuan skalabilitas. Pada empat worker dan rentang 100–500 task, RR memberi makespan sekitar setengah MET pada setiap ukuran beban, throughput sekitar dua kali MET, DI jauh lebih rendah, dan utilisasi rata-rata lebih tinggi. Makespan kedua algoritma naik kira-kira sebanding dengan total beban, sedangkan throughput mendatar. Karena MET dalam implementasi ini selalu memilih vm-worker-1, hasilnya tidak menunjukkan perbaikan apa pun dari penambahan worker.

5.4 Keterbatasan dan Ancaman Validitas

- Baseline yang dibahas hanya satu run per algoritma; tidak cukup untuk menyatakan hasil signifikan secara statistik.
- Run berlabel load_300 pada baseline masih berisi 100 task dan tidak dipakai untuk analisis skalabilitas; analisis skalabilitas memakai eksperimen terpisah berlabel scale_* (Bagian 5.3).
- Ukuran SKU VM dan dokumentasi resize belum tersedia dalam artefak yang diberikan.
- Failover Azure belum menghentikan VM target MET; uji Azure hanya menghentikan vm-worker-4.
- Utilisasi yang dilaporkan adalah utilisasi berbasis waktu sibuk, bukan CPU% dari Azure Monitor.
- Worker diakses melalui IP publik pada port 8000; ini berbeda dari rencana IP privat dan memerlukan verifikasi aturan keamanan.
- Run awal dengan tiga worker dikecualikan dari baseline empat worker; hasilnya tidak boleh digabung tanpa penjelasan.
- Peringatan master menyebut estimasi makespan berada di luar rentang 30–180 detik: workload baseline hanya berlangsung sekitar 5–12 detik, dan pada eksperimen skalabilitas peringatan muncul untuk beban 100 dan 200 task (estimasi sekitar 11 dan 23 detik) tetapi tidak untuk 300–500 task. Hasil pada beban kecil dapat lebih dipengaruhi overhead komunikasi, kalibrasi, dan orkestrasi.

- Eksperimen skalabilitas hanya memiliki tiga pengulangan untuk setiap kombinasi jumlah task dan algoritma; SD dari tiga nilai tidak stabil dan tidak cukup untuk klaim signifikansi statistik.
- Workload skalabilitas adalah pemrosesan gambar JPEG sintetis, bukan GoCJ; hasilnya tidak dapat dibandingkan langsung dengan hasil CloudSim sebelumnya.
- Dataset yang sama (seed 42) dipakai untuk semua run, dan beban N task adalah N task pertama pada manifest; variasi yang diukur adalah variasi runtime, bukan variasi dataset, dan komposisi ukuran gambar berbeda antar ukuran beban (rata-rata length per task 135,94–142,67), yang dapat memengaruhi throughput antar ukuran.
- Urutan eksekusi tidak diacak: beban menaik dan MET selalu dijalankan sebelum RR pada setiap ukuran beban dalam setiap pengulangan. Efek urutan atau kondisi VM yang berubah seiring waktu tidak diuji.
- Kapasitas kalibrasi diukur ulang pada setiap run dari waktu dinding di master sehingga dapat mencakup transfer dan latensi jaringan; keputusan MET sangat bergantung pada nilai ini dan pada toleransi seri 5%.
- avg_wait_s adalah rata-rata send_s (waktu relatif pengiriman), bukan waktu antre konvensional; utilisasi dihitung dari exec_s dan 2 core per worker, bukan CPU% dari Azure Monitor. Tidak ada pengukuran Azure Monitor, jaringan, maupun biaya pada lampiran.
- Ukuran/SKU VM, RAM, region, Resource Group, konfigurasi NSG, biaya, dan pemisahan host fisik tidak dapat diverifikasi dari bukti yang tersedia dan ditandai belum terverifikasi; pengelompokan Host A/Host B adalah label arsitektur proyek.
- Pada beban kecil (100–200 task) hasil lebih rentan terhadap overhead komunikasi, kalibrasi, dan orkestrasi dibanding pada beban lebih besar.

5.5 Kesimpulan dan Saran

Pada baseline 100 task, MET berhasil dijalankan pada Azure dan menunjukkan pola yang sama seperti simulasi sebelumnya: seluruh task diarahkan ke satu VM yang dianggap terbaik, sehingga DI mencapai jumlah worker (4,0). Round Robin menghasilkan makespan lebih rendah dan throughput lebih tinggi karena memanfaatkan empat worker.

Eksperimen skalabilitas pada rentang 100–500 task (dataset gambar sintetis, empat worker, tiga pengulangan per kombinasi) menguatkan pola baseline. Makespan MET naik dari 11,74 s menjadi 56,81 s dan RR dari 5,77 s menjadi 28,51 s, sehingga RR sekitar dua kali lebih cepat (rasio 1,97–2,04) pada semua ukuran beban yang diuji. Throughput kedua algoritma relatif mendatar (MET sekitar 8,4–8,8 task/s; RR sekitar 16,5–17,5 task/s). DI MET tetap 4,000 pada semua beban karena seluruh task masuk ke vm-worker-1, sedangkan DI RR sekitar 0,57–0,65. Utilisasi rata-rata berbasis exec_s sekitar 24% untuk MET dan sekitar 42% untuk RR.

Kesimpulan ini dibatasi pada konfigurasi yang diuji: empat worker, beban 100–500 task, workload pemrosesan gambar sintetis (bukan GoCJ), dan implementasi MET dengan toleransi seri 5% tanpa memperhitungkan antrean. Hasil tidak membuktikan bahwa RR selalu lebih baik daripada MET pada workload, jumlah worker, atau lingkungan lain, dan tidak dapat digeneralisasi di luar 500 task. Karena hanya tiga pengulangan pada dataset yang sama, tidak ada klaim signifikansi statistik.

Saran: (1) menambah pengulangan dan mengacak urutan beban serta urutan algoritma; (2) menguji lebih dari satu dataset/seed; (3) membandingkan dengan varian MET yang memperhitungkan antrean atau beban; (4) mengukur CPU% melalui Azure Monitor dan latensi jaringan secara terpisah; (5) mendokumentasikan SKU VM, region, dan konfigurasi jaringan dari portal Azure.

Lampiran A. Cara Membuat Grafik

Dari direktori repo, jalankan:

python plot_results.py --results results

Dependensi: pandas, matplotlib, numpy. Skrip menghasilkan results/figures/, results/hasil_ringkasan.md, results/hasil_agregat.csv, dan results/catatan_grafik.md. Untuk memasukkan run non-failover dengan kurang dari empat worker, gunakan --include-partial. Jangan menafsirkan label sebagai jumlah task; selalu gunakan kolom n_tasks.

Lampiran B. Berkas Hasil Skalabilitas

Arsip hasil_scalability_500.tar.gz berisi folder results_scalability_500 dan skrip plot_scalability_500.py. Isi yang diperiksa: summary.csv (30 baris data), 30 berkas tasks_<ALGO>_<timestamp>.csv, run.log, run.pid, scalability_mean_sd.csv (50 baris data), dan lima grafik pada folder figures (makespan_vs_task.png, waiting_time_vs_task.png, throughput_vs_task.png, imbalance_vs_task.png, utilization_vs_task.png). Arsip tidak memuat kode proyek lain maupun dataset gambar.

Grafik dan tabel agregat dibuat dengan plot_scalability_500.py, yang membaca summary.csv pada folder results_scalability_500, hanya memakai label berawalan scale_, memvalidasi bahwa tiap kombinasi jumlah task dan algoritma memiliki tepat tiga label rep1–rep3, lalu menghitung mean dan simpangan baku sampel (statistics.stdev) dan menggambar mean ± sample SD. Grafik pada laporan ini adalah keluaran langsung skrip tersebut.